

Design Typeahead

Google · Amazon · YouTube – "queries లెక్కపెట్టి, trie lo పెట్టి, prefix కి top-10 ఇద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – ఇప్పుడే వైరల్ అయిన query **28 గంటల తర్వాత** కనిపిస్తుంది, మరియు దాన్ని సరిచేయడం CPU ని **96 రెట్లు** పెంచుతుంది. రెండోది – personalization జోడిస్తే cache hit rate **95.1% నుంచి 0.2%** కి పడిపోతుంది, మరియు cache ని **100 రెట్లు** పెంచినా అది మారదు. మూడోది – అరుదైన prefixes కి మీ "top-10" lo **అరు రోజు మారతాయి**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

LLD DEEP 17 (AUTOCOMPLETE) LO TRIE ఉంది - దాన్ని ఇక్కడ మళ్ళీ రాయను

అక్కడ `Node`, `cached top[K]`, `sorted` జాబితాల `merge`, `root-నుంచి-word` దారిలో `#recompute` - అన్నీ కొలిచాం. "ఒక trie lo top-K ఎలా ఉంచాలి" అనే ప్రశ్నకి జవాబు అక్కడ పూర్తిగా ఉంది.

ఈ doc lo **ఒక్క trie node కూడా లేదు**. ఎందుకంటే HLD స్థాయిలో trie సమస్య కాదు - నేను కొలిచాను: 50 కోట్ల queries యొక్క index **0.6 TB**, అంటే రెండు machines. అది ఒక engineering పని, ఒక design సవాలు కాదు.

నిజమైన సవాళ్ళు మూడు, మరియు మూడూ trie బయట ఉన్నాయి:

ఆ index ఎప్పుడు తాజా అవుతుంది (§5) · దాన్ని cache చేయగలమా (§8) · అసలు ఆ లెక్కలు నమ్మదగినవా (§11).

మరియు ఒక విషయం ముందే చెప్తాను: నేను ఈ doc lo **రెండు కొలతలు చేసి, రెండూ "సమస్య లేదు" అని తేల్చాను** - index పరిమాణం, మరియు servers మధ్య top-K కలపడం. ఆ రెండు ప్రతికూల ఫలితాలూ §1 lo ఉన్నాయి, ఎందుకంటే **ఎక్కడ సమస్య లేదో తెలియడం కూడా ఒక ఫలితం**.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏవి సమస్యలు కావు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: index ఎప్పుడు తాజా అవుతుంది

3. Step – రాత్రి ఒక పెద్ద batch job
4. మొదటి విరుపు – 28 గంటలు, లేదా 96 రెట్లు CPU
5. Step – రెండు అంచెలు

Part 3 – రెండో విరుపు: personalization

6. Step – ప్రతి keystroke కి ఒక request
7. రెండో విరుపు – 95.1% నుంచి 0.2%
8. Step – global cache, local క్రమం

Part 4 – మూడో విరుపు: నమ్మదగని లెక్కలు

9. Step – prefix కి top-10
10. మూడో విరుపు – పదింటిలో ఆరు రోజు మారతాయి
11. Step – సాక్ష్యం లేకపోతే మౌనం

Part 5 – పూర్తి system

12. మొత్తం code · 12 లక్షల queries · mutation testing

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటితో చెప్పాల్సిన English script

15. Follow-ups – అక్షర దోషాలు, భాషలు, చెడు queries

16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏవి సమస్యలు కావు

వాడుకరి "how t" అని టైప్ చేస్తాడు. 50 మిల్లీసెకన్లలో పది సూచనలు కనిపించాలి. ప్రతి keystroke కీ మళ్ళీ.

ఇది ఒక **read-heavy, latency-critical** problem. ఆచరణలో ప్రతి search కి 10-20 typeahead requests.

Design మొదలుపెట్టే ముందు, **ఏవి సమస్యలు కావో** తెలుసుకుందాం – ఎందుకంటే interview lo చాలా సమయం వాటి మీదే వృథా అవుతుంది.

కాదు: index చాలా పెద్దది

ప్రత్యేక queries : 50 కోట్లు · సగటు పొడవు 22 అక్షరాలు
ఒక్కో node : 60 bytes నిర్మాణం + 300 bytes top-10 cache = 360 bytes
prefixes 15% ప్రత్యేకం → 1.6 బిలియన్ nodes → 0.6 TB
512 GB machines : 2 కావాలి

రెండు machines. అది ఒక design సవాలు కాదు.

కాదు: servers మధ్య top-K కలపడం

Queries వేల servers కి వస్తాయి. ప్రతి server తన **top-100 మాత్రమే** పంపితే, global top-100 సరిగ్గా వస్తుందా? నేను భయపడ్డాను. కలిచాను:

servers	పంపిన entries	నిజమైన top-100 lo ఎన్ని దొరికాయి
1	100	100%
10	1,000	100%

100	10,000	99%
1000	1,00,000	100%

1,000 servers తో కూడా 100%. ఎందుకంటే ప్రజాదరణ పొందిన queries ప్రతి server మీదా ప్రజాదరణ పొందుతాయి – అవి ఏ ఒక్క server యొక్క top-100 నుంచే జారిపోవు.

ఈ రెండు ప్రతికూల ఫలితాలూ ఎందుకు ముఖ్యం

Interview lo చాలామంది ఈ రెండింటి మీదా 20 నిమిషాలు గడుపుతారు – sharding, Count-Min Sketch, HyperLogLog.

కొలిన్స్: index 2 machines lo పడుతుంది, మరియు అమాయకమైన top-K కలపడం 100% సరైనది.

ఒక సంఖ్య వేయడం ద్వారా మీరు 20 నిమిషాలు ఆదా చేసి, నిజమైన సమస్యల మీద గడవగలరు.

నిజమైన సమస్యలు మూడు, మరియు అవి సమయం, cache, మరియు నమ్మకం గురించి.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
కొత్త/trending queries ఎంత త్వరగా కనిపించాలి?	ఇదే §4 మొత్తం – గంటలా, నిమిషాలా
సూచనలు ఒక్కొక్కరికీ వేరుగా ఉండాలా?	అవును అంటే cache 95% నుంచి 0.2% – §7
Latency బడ్జెట్ ఎంత?	50 ms అంటే cache తప్పనిసరి, అంటే §7 ఒక అడ్డంకి
అరుదైన prefixes కి ఏమి చూపించాలి?	§10 – తగినంత డేటా లేకపోతే సూచనలు noise
అక్షర దేవులు సరిచేయాలా?	ఇది trie ని పూర్తిగా మారుస్తుంది – §15
ఎన్ని భాషలు, ఎన్ని లిపులు?	ఒక్కే లిపికి ఒక్కే index – §15
ఒక query ని తీసేయాల్సి వస్తుందా?	అవును (అసభ్యం, పరువునష్టం) – అది ఒక అత్యవసర దారి కావాలి

ఒక ప్రశ్న మీ మొత్తం ARCHITECTURE ని నిర్ణయిస్తుంది

"Trending queries ఎంత త్వరగా కనిపించాలి?"

"రోజుకి ఒకసారి చాలు" అంటే – ఒక్క batch pipeline సరిపోతుంది, మరియు design 20 నిమిషాల్లో ముగుస్తుంది.

"నిమిషాల్లో" అంటే – మీకు రెండు వేర్వేరు systems కావాలి (§5), మరియు వాటిని query సమయంలో కలపాలి.

ఇది ఒక feature ప్రశ్న లాగా అనిపిస్తుంది. ఇది నిజానికి architecture ప్రశ్న.

Part 2 – మొదటి విరుపు: index ఎప్పుడు తాజా అవుతుంది

3. Step – రాత్రి ఒక పెద్ద batch job

Typeahead index ఎలా తయారవుతుంది? రోజంతటి search logs మీద ఒక పెద్ద job:

logs (రోజంతా) → queries లెక్కపెట్టు → prefix → top-K → index → serving machines

ఇది సహజమైన, సరైన నిర్మాణం. Logs పెద్దవి కాబట్టి job గంటలు పడుతుంది – అనుకుందాం **6 గంటలు, 400 CPU-గంటలు**.

రోజుకి ఒకసారి నడుపుతాం. సరిపోతుందా?

4. మొదటి విరుపు – 28 గంటలు, లేదా 96 రెట్లు CPU

ఒక సంఘటన జరిగింది – ఒక నటుడు మరణించాడు, ఒక ఫలితం వచ్చింది, ఒక ఉత్పత్తి విడుదలైంది. లక్షలమంది ఒక కొత్త query టైప్ చేస్తున్నారు.

అది typeahead lo ఎప్పుడు కనిపిస్తుంది?

index ఎంత తరచుగా build ఎంత సేపు ఎంత ఆలస్యంగా కనిపిస్తుంది		
24 గం	6 గం	28.0 గంటలు
6 గం	2 గం	6.0 గంటలు
1 గం	40 ని	40 నిమిషాలు
15 ని	12 ని	12 నిమిషాలు

రోజుకి ఒక BUILD → 28 గంటలు ఆలస్యం

కారణం రెండు భాగాలు: తర్వాతి build మొదలవడానికి ఎదురుచూపు (**22 గంటలు**), ఆ తర్వాత build నడవడానికి (**6 గంటలు**).

అంటే ఆ సంఘటన **ముగిసిపోయిన తర్వాత** మీ typeahead దాన్ని సూచించడం మొదలుపెడుతుంది.

మరియు typeahead యొక్క అత్యంత విలువైన క్షణం సరిగ్గా అదే – **వాడుకరికి ఏమి వెతకాలో ఇంకా తెలియనప్పుడు**.

సహజమైన పరిష్కారం: **తరచుగా rebuild చెయ్యడం**. దాని ఖర్చు?

index ఎంత తరచుగా రోజుకి builds CPU-గంటలు/రోజు (ఒక build = 400 CPU-గం)		
24 గం	1	400
6 గం	4	1,600
1 గం	24	9,600
15 ని	96	38,400

12 నిమిషాల తాజాదనం కోసం 96 రెట్లు CPU. మరియు ప్రతి build **అదే 50 కోట్ల queries** ని మళ్ళీ మళ్ళీ లెక్కపెడుతోంది – వాటిలో 99.99% మారలేదు.

5. Step – రెండు అంచెలు

ఇక్కడ అసలు అంతర్దృష్టి ఉంది, మరియు అది ఒక్క వాక్యంలో:

పాత డేటా పెద్దది కానీ స్థిరం • కొత్త డేటా చిన్నది కానీ చంచలం

మీ index lo 50 కోట్ల queries ఉన్నాయి, మరియు అవి నిన్నటికీ ఈరోజుకీ దాదాపు ఒకటే.

మారేది గత కొన్ని నిమిషాల డేటా మాత్రమే – అది కొన్ని వేల queries.

కాబట్టి **ఒకే index కి బదులు రెండు**: ఒక పెద్ద స్థిరమైనది (రోజుకి ఒకసారి), ఒక చిన్న చంచలమైనది (నిరంతరం). మరియు వాటిని **query సమయంలో** కలపడం.

```
// §5 – నిజ-సమయ పొర. ఇది ఒక పెద్ద build కోసం ఎదురుచూడదు.
record(query, n = 1) {
  if (typeof query !== 'string' || !query.length) return false;
  this.#realtime.set(query, (this.#realtime.get(query) || 0) + n);
  this.stats.recorded += n;
  return true;
}
```

చదివేటప్పుడు రెండింటినీ కలుపుతాం:

```
// నిజ-సమయ పొర నుంచి – ఇది చిన్నది కాబట్టి నేరుగా చూడొచ్చు
let rt = 0;
for (const [q, n] of this.#realtime) {
  if (!q.startsWith(prefix)) continue;
  seen.set(q, (seen.get(q) || 0) + n);
  rt++;
}
```

ఆ **for** చూసి "ఇది $O(n)$ scan, ఘోరం" అనిపించొచ్చు. కానీ **#realtime చిన్నది** – గత 15 నిమిషాల queries మాత్రమే. అది కొన్ని వేలు, మరియు memory lo. పెద్ద index కి index ఉంది; చిన్నదానికి అవసరం లేదు.

1 నిమిషం తాజాదనం • 400 CPU-గంటలు/రేజి

96 రెట్లు CPU ఖర్చుతో 12 నిమిషాలు సాధించడానికి బదులు – **అదే ఖర్చుతో 1 నిమిషం.**

ఎందుకంటే మనం "తాజాదనం" మరియు "పరిపూర్ణత" అనే రెండు వేర్వేరు అవసరాలని **రెండు వేర్వేరు systems** కి అప్పగించాం – ఒక్కటే system రెండింటినీ చేయమని బలవంతం చేయకుండా.

మరియు ఒక build ముగిసినప్పుడు చిన్నది పెద్దదానిలో కలిసిపోతుంది:

```
// ఒక build జరిగింది – నిజ-సమయ పొరని offline lo కలిపేయడం
promote() {
  for (const [q, n] of this.#realtime) {
    this.#offline.set(q, (this.#offline.get(q) || 0) + n);
    this.#index(q);
    this.stats.promoted++;
  }
  this.#realtime.clear();
}
```

Part 3 – రెండో విరుపు: personalization

6. Step – ప్రతి keystroke కి ఒక request

Typeahead ప్రతి అక్షరానికి పిలవబడుతుంది. ఒక search = 10-20 requests. Latency బడ్జెట్ **50 ms**.

ఆ రెండూ కలిపి ఒకటే చెప్తున్నాయి: **cache తప్పనిసరి**.

మరియు cache చేయడం సులభం – `prefix → suggestions`. Prefixes Zipf పంపిణీలో ఉంటాయి ("how", "wea", "you" చాలా సాధారణం), కాబట్టి hit rate ఎక్కువ.

ఇప్పుడు product team అడుగుతుంది: "సూచనలు వాడుకరిని బట్టి ఉండొచ్చా?" – అతని గత searches, అతని ప్రాంతం, అతని భాష.

అది సహేతుకమైన కోరిక. దాని ఖర్చు కొలుద్దాం.

7. రెండో విరుపు – 95.1% నుంచి 0.2%

Typeahead cache · ప్రతి keystroke ఒక request · 30 లక్షల entries cache

విధానం	cache key	hit rate	backend కి వెళ్ళేవి
అందరికీ ఒకే suggestions	prefix	95.1%	4.9%
ఒక్కొక్కరికీ వేరే	prefix + user	0.2%	99.8%

95.1% → 0.2% · BACKEND భారం 20 రెట్లు

Cache key `prefix` నుంచి `prefix + user` కి మారింది. అంతే.

కానీ key space **2 లక్షల prefixes** నుంచి **2 లక్షలు × 2 కోట్ల users** కి పెరిగింది. ఏ cache అయినా అందులో ఒక చుక్క మాత్రమే.

Backend కి వెళ్ళేవి **4.9% నుంచి 99.8%** – అంటే మే serving fleet **20 రెట్లు** పెద్దది కావాలి.

"అయితే cache పెద్దది చేద్దాం" అనేది సహజమైన స్పందన. కొలుద్దాం:

cache entries	personalized hit rate
3 మి	0.2%
30 మి	0.2%
300 మి	0.2%

CACHE ని 100 రెట్లు పెంచాం. HIT RATE ఒక్క దశాంశం కూడా మారలేదు.

ఎందుకంటే సమస్య cache పరిమాణం కాదు – **key space యొక్క ఆకారం.**

ఒక్కో (prefix, user) జత ఆ వాడుకరి జీవితంలో **ఒకటి రెండు సార్లు** మాత్రమే వస్తుంది. Cache lo ఉంచినా అది **మళ్ళీ అడగబడదు.**

ఏదీ పునరావృతం కానప్పుడు cache నిరుపయోగం – అది ఎంత పెద్దదైనా.

8. Step – global cache, local క్రమం

పరిష్కారం personalization ని వదులుకోవడం కాదు. అది **ఎక్కడ జరుగుతుందో** మార్చడం.

Personalization ఒక వడపోత కాదు, ఒక పునఃక్రమం. వాడుకరికి కనిపించాల్సిన సూచనలు దాదాపు అందరికీ ఒకటే – మారేది వాటి **క్రమం.**

కాబట్టి:

1. Backend **global top-20** ఇస్తుంది → cache key ఇంకా **prefix** → **95.1% hit rate**
2. ఆ 20 ని వాడుకరి చరిత్రతో **తిరిగి క్రమం** పెట్టి top-10 చూపించడం
3. ఆ పునఃక్రమం **client మీద** (లేదా ఒక చవక్తైన edge step lo) జరుగుతుంది

విధానం	cache hit	personalization ఉందా
global cache + client-side rerank	95.1%	అవును

ఒక సాధారణ నమూనా – ఖరీదైన భాగాన్ని పంచుకోండి, చవక్తైన భాగాన్ని వ్యక్తిగతం చేయండి

ఖరీదైనది: 50 కోట్ల queries lo ఈ prefix కి ఏవి ప్రజాదరణ పొందాయో కనుక్కోవడం. అది **అందరికీ ఒకటే.**

చవక్తైనది: 20 అంశాలని ఒక వాడుకరి చరిత్రతో క్రమం పెట్టడం. అది **ఒక్కొక్కరికే వేరే**, కానీ అది **మైక్రోసెక్షన్ల పని.**

ఈ విభజన లేకపోతే మీరు ఖరీదైన భాగాన్ని కూడా వ్యక్తిగతం చేసి, cache ని పోగొట్టుకుంటారు.

ఒక personalization · రెండు చట్ట · పూర్తిగా వేరే ఖర్చు

backend lo personalize

cache key: prefix + user

hit rate 0.2% · backend 20x పెద్దది

client lo personalize

cache key: prefix · top-20 తెచ్చి క్రమం

hit rate 95.1% · backend అదే

రెండింటిలోనూ వాడుకరికి **ఒకే అనుభవం.**

తేడా – ఏ పనిని **పంచుకున్నాం**, ఏ పనిని **విడదీశాం.**

"ఏవి సంబంధితమైనవి" – అందరికీ ఒకటే · ఖరీదు · **cache చెయ్యి**

"ఏది ముందు" – ఒక్కొక్కరికే వేరే · చవక్త · **చివర్లో చెయ్యి**

Part 4 – మూడో విరుపు: నమ్మదగని లెక్కలు

9. Step – prefix కి top-10

ఇప్పుడు index తాజాగా ఉంది, cache పని చేస్తోంది. మిగిలినది సులభం: ఒక prefix కి, ఆ prefix తో మొదలయ్యే queries లో **అత్యధికంగా వచ్చిన 10**.

ప్రజాదరణ పొందిన prefixes కి ఇది బాగా పని చేస్తుంది – "how t" కి లక్షల queries, top-10 స్పష్టం.

కానీ మీ index లో చాలావరకు prefixes **అరుదైనవి**. "xyl", "qwa", ఒక తప్పు అక్షరంతో మొదలయ్యేవి. వాటికి?

10. మూడో విరుపు – పదింటిలో ఆరు రోజు మారతాయి

ఒక prefix కింద 60 అభ్యర్థులు · 30 రోజులు · top-10 రోజురోజుకీ ఎంత మారుతుంది?

ఆ prefix కి రోజువారీ queries | రోజుకి మారే suggestions (10 లో)

10,00,000	0.0
10,000	0.6
500	1.9
60	4.3
15	5.9

రోజుకి 15 QUERIES ఉన్న PREFIX కి – పదింటిలో ఆరు రోజు మారతాయి

అంటే ఆ "top-10" ఒక ప్రజాదరణ కొలత కాదు – అది ఒక **యాదృచ్ఛిక నమూనా**.

వాడుకరి ఈరోజు చూసినది రేపు ఉండదు. అతనికి typeahead **చంచలంగా, నమ్మదగనిదిగా** కనిపిస్తుంది.

మరియు మొదటి వరుస చూడండి – లక్షల queries ఉన్న prefix కి churn **సున్నా**. కాబట్టి ఇది ఒక algorithm లోపం కాదు; ఇది **సాక్ష్యం లేకపోవడం**.

11. Step – సాక్ష్యం లేకపోతే మౌనం

పరిష్కారం ఒక మంచి algorithm కాదు. **తక్కువ సాక్ష్యంతో మంచి సమాధానం ఇవ్వడం అసాధ్యం** – కాబట్టి సమాధానం ఇవ్వకూడదు.

```
const out = [...seen.entries()]
  .filter(([, n]) => n >= this.minCount) // $11 - కనీస సాక్ష్యం
  .sort((a, b) => b[1] - a[1] || (a[0] < b[0] ? -1 : 1))
  .slice(0, k);
if (!out.length) this.stats.suppressed++;
```

రోజువారీ queries | కనీస పరిమితి లేదు | కనీసం 5 | 30 రోజుల్లో ఎన్ని రోజులు చూపించాం

10,000		0.6		0.6 30/30
500		1.9		1.9 30/30
60		4.3		చూపించలేదు 0/30
15		6.0		చూపించలేదు 0/30

చివరి రెండు వరుసలు - ఏమీ చూపించలేదు, మరియు అదే సరైన పని

ఖాళీ typeahead ఒక వైఫల్యం లాగా కనిపిస్తుంది. కానీ ప్రత్యామ్నాయం - **రోజూ మారే యాదృచ్ఛిక పది** - ఇంకా ఘోరం.

వాడుకరి ఖాళీ typeahead చూస్తే అతను **ఫైవ్ చేయడం కొనసాగిస్తాడు**. చంచలమైన సూచనలు చూస్తే అతను **వాటిని నమ్మడం మానేస్తాడు** - మరియు అది మీ ప్రజాదరణ పొందిన prefixes ని కూడా దెబ్బతీస్తుంది.

ఒక నమ్మదగని సమాధానం, సమాధానం లేకపోవడం కంటే ఖరీదైనది.

Part 5 – పూర్తి system

12. మొత్తం code • 12 లక్షల queries • mutation testing

ఆరు నియమాలు

#	నియమం	విరిగితే అర్థం
1	ప్రతి సూచనా prefix తో మొదలవ్వాలి	index తప్పు
2	తగ్గుతున్న క్రమంలో	క్రమం విరిగింది
3	Duplicates లేవు	రెండు అంచెలు నకలు అయ్యాయి
4	ప్రతిదీ minCount దాటాలి	§11 రక్షణ లేదు
5	చూపించిన లెక్క నిజమైన లెక్కతో సరిపోవాలి	రెండు అంచెల కలయిక తప్పు
6	ఉండాల్సిన top-k తప్పిపోకూడదు	తాజాదనం విరిగింది

నియమం 5 ఒక నిజమైన bug ని పట్టుకుంది

మొదటి పరుగులో 1,000 lo 1 విఫలమైంది:

ఉదా (seed 854): "bddc": చూపించినది 3 ≠ నిజం 4

కారణం: నా prefix index maxPrefixLen వరకే ఉంది (memory ఆదా కోసం). అంతకంటే పొడవైన prefix అడిగితే:

- offline దారి: #prefix.get("bddc") → ఏమీ లేదు
- నిజ-సమయ దారి: "bddc".startsWith("bddc") → ఉంది

కాబట్టి offline లెక్క (1) అదృశ్యమై, నిజ-సమయ లెక్క (3) మాత్రమే కనిపించింది.

ఇది "ఫలితం రాలేదు" కాదు – ఇది తప్పు సంఖ్య

Index truncation ఒక సహేతుకమైన memory optimisation. దాని ఆశించిన ఫలితం: "పొడవైన prefixes కి సూచనలు రావు".

దాని అసలు ఫలితం: పొడవైన prefixes కి సూచనలు వస్తాయి, కానీ తప్పు లెక్కతో – మరియు ఆ తప్పు లెక్క క్రమాన్ని మారుస్తుంది.

మౌనంగా తప్పు జవాబు ఇవ్వడం, జవాబు ఇవ్వకపోవడం కంటే ఎప్పుడూ ఘోరం.

పరిష్కారం: పొడవైన prefix కి, పొట్టి prefix యొక్క అభ్యర్థులని తీసి పూర్తి prefix తో వడపోయడం:

```
// ముఖ్యం: index maxPrefixLen వరకే ఉంది. అంతకంటే పొడవైన prefix అడిగితే
// index lo నేరుగా దొరకదు - కాబట్టి *పొట్టి* prefix యొక్క అభ్యర్థులని తీసి
// పూర్తి prefix తో వడపోయాలి. లేకపోతే offline లెక్క కనిపించకుండా పోయి,
// నిజ-సమయ లెక్క మాత్రమే మిగిలి - *తప్పు సంఖ్య* చూపిస్తాం. §13 చూడండి.
const lookup = prefix.length <= this.maxPrefixLen
  ? prefix : prefix.slice(0, this.maxPrefixLen);
for (const q of this.#prefix.get(lookup) || []) {
  if (!q.startsWith(prefix)) continue;
  seen.set(q, (this.#offline.get(q) || 0));
}
```

1,000 యాదృచ్ఛిక ప్రయోగాలు · 12,35,176 queries · 3,42,285 suggest calls
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

recorded	12,35,176
suggested	3,42,285
fromRealtime	73,187
suppressed	1,85,074
promoted	4,65,058
merged	1,51,167

suppressed 1,85,074 – అంటే §11 యొక్క మౌనం 3.4 లక్షల calls lo 1.85 లక్షల సార్లు పని చేసింది.

సగానికి పైగా prefixes కి సరైన జవాబు "ఏమీ లేదు".

Test విఫలం కాగలదా? – నాలుగు మార్పులు

మార్పు లేని code	→	0/1000 విఫలం
నిజ-సమయ పొరని కలపకపోతే	→	300/300 విఫలం
ఉదా: "d": చూపించినది 9 ≠ నిజం 10		
కనీస సాక్ష్యం పరిమితి తీసేస్తే	→	259/300 విఫలం
ఉదా: "beceda" లెక్క 3 < కనీసం 6		
క్రమం పెట్టకపోతే	→	300/300 విఫలం
ఉదా: క్రమం తప్పింది: 11 తర్వాత 18		
promote తర్వాత నిజ-సమయ పొరని ఖాళీ చెయ్యకపోతే	→	300/300 విఫలం
ఉదా: "d": చూపించినది 19 ≠ నిజం 10		

చివరి మార్పు ముఖ్యమైనది: `promote()` తర్వాత నిజ-సమయ పొరని ఖాళీ చేయకపోతే, ఆ queries

రెండుసార్లు లెక్కవృత్తాయి (offline lo ఒకసారి, realtime lo మళ్ళీ) – **19 ≠ 10**. **రెండు అంచెల designs lo ఇది అత్యంత సాధారణమైన bug.**

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి
0-6	ఏవి సమస్యలు కావ్ ముందే తేల్చండి – index 0.6 TB , top-K కలపడం 100% . ఇది మిమ్మల్ని వేరుగా చూపిస్తుంది
6-10	§2 ప్రశ్నలు. ముఖ్యంగా "trending ఎంత త్వరగా?"
10-20	మొదటి విరుపు. 28 గంటలు vs 96x CPU. రెండు అంచెలు → 1 నిమిషం, అదే ఖర్చు
20-30	రెండో విరుపు. 95.1% → 0.2%, మరియు cache 100x పెంచినా అదే. Client-side rerank
30-38	మూడో విరుపు. అరుదైన prefixes కి 10 lo 6 రోజు మారతాయి. మౌనమే సరైన జవాబు
38-45	ఆరు నియమాలు, మరియు index truncation bug

ఏమి తప్పక చెప్పాలి

1. **Index పరిమాణం మరియు top-K కలపడం ఈ problem యొక్క సమస్యలు కావు.** సంఖ్యలతో చెప్పండి.
2. **తాజాదనం మరియు పరిపూర్ణత రెండు వేర్వేరు అవసరాలు** – వాటిని రెండు systems కి అప్పగించండి.
3. **Personalization ఒక వడపోత కాదు, ఒక పునఃక్రమం.** ఖరీదైన భాగాన్ని పంచుకోండి.
4. **Cache ని పెద్దది చేయడం key space సమస్యని పరిష్కరించదు.**
5. **తక్కువ సాక్ష్యం ఉన్నప్పుడు మౌనమే సరైన జవాబు.**

ఏమి వదిలేయాలి

- Trie అమలు వివరాలు – LLD Deep 17 lo ఉన్నాయి, ఒక వాక్యం చాలు
- Count-Min Sketch / HyperLogLog – §1 చూపించినట్టు ఇక్కడ అవసరం లేదు
- Ranking model యొక్క features – ఇది ఒక ML ప్రశ్న
- Prefix compression (radix tree) – ఒక optimisation, ఒక design కాదు

14. నీటితో చెప్పాల్సిన English script

"Before designing, I want to rule two things out with numbers, because interviews usually spend twenty minutes on them. First, index size: 500 million unique queries at 22 characters, with a cached top-10 per node, is about **0.6 TB** – two machines. Not a design problem. Second, distributed top-K: if queries hit a thousand servers and each sends only its own top-100, do we still get the true global top-100? I measured it – **100%**, because a query popular globally is popular on every shard. So no sketches needed. **Knowing where the problem isn't is a result too.**"

"The real problems are freshness, caching, and confidence."

"Freshness first. The natural design is a nightly batch over the day's logs – say six hours of work. When something goes viral, I measured the lag: the next build starts 22 hours later and takes 6, so the query appears **28 hours** after the event. That's after the event is over, and typeahead's most valuable moment is exactly when users don't yet know what to search for. Rebuilding every 15 minutes gets it to 12 minutes – at **96x the CPU**, re-counting 500 million queries that haven't changed."

"The fix is noticing that **old data is large but stable, and new data is small but volatile**. So: two indexes. A big offline one rebuilt daily, and a small real-time layer that's just the last few minutes, merged at query time. That gives **one-minute freshness at the original 400 CPU-hours**. The real-time layer can be a linear scan – it's a few thousand entries in memory. The big index needs an index; the small one doesn't."

"Second, caching. Typeahead runs on every keystroke with a 50ms budget, so caching is mandatory, and `prefix → suggestions` gets **95.1%** hits. Then product asks for personalization. Changing the key to `prefix + user` drops it to **0.2%** – backend load goes from 4.9% to 99.8%, so you need twenty times the serving fleet. And the instinct to grow the cache doesn't work: I measured 3 million, 30 million, and 300 million entries, and the hit rate stayed at **0.2%** in all three. The problem isn't cache size, it's that **each (prefix, user) pair occurs once or twice in that user's lifetime** – nothing repeats, so nothing caches."

"The fix is to notice that personalization is a **re-ranking, not a filter**. Which suggestions are relevant is almost the same for everyone; only the order differs. So the backend returns a cached global top-20 – still 95.1% – and the client reorders it using local history. **Share the expensive part, personalize the cheap part.**"

"Third, confidence. For a prefix with a million queries a day, the top-10 is rock stable – zero changes day to day. For one with fifteen queries a day, I measured **six of the ten change every day**. That's not a ranking, it's a random sample, and the user sees typeahead as unreliable. The fix isn't a better algorithm – with that little evidence there is no good answer – so I require a minimum count and **show nothing** when it isn't met. An empty dropdown looks like a failure, but a suggestion list that changes daily is worse: it teaches users not to trust the feature at all, including where it works."

"Six invariants, a thousand runs, 1.2 million queries. The fuzz failed **1 in 1,000** with a real bug, and its shape is worth describing. I truncate the prefix index at a maximum length to save memory, expecting that longer prefixes simply return nothing. What actually happened: a long prefix missed the offline index but still matched the real-time layer, so it returned suggestions with **only the real-time count** – a silently wrong number, which changes the ordering. **Failing to answer is fine; answering wrongly is not**. The fix is to look up the longest indexed prefix and filter by the full one."

15. Follow-ups – అక్షర దోషాలు, భాషలు, చెడు queries

"వాడుకరి 'wheather' అని టైప్ చేస్తే?"

Prefix trie ఇది పట్టుకోదు – దానికి edit distance కావాలి. ఆచరణలో రెండు దారులు: (1) **query logs నుంచే** నేర్చుకోవడం – చాలామంది "wheather" అని టైప్ చేసి తర్వాత "weather" ని click చేస్తే, ఆ జతని ఒక మార్పుగా నిల్వ చేయడం (ఇది model అవసరం లేని, చాలా బలమైన సంకేతం), లేదా (2) **BK-tree / trie మీద fuzzy walk** – కానీ అది ఖరీదు, మరియు 50 ms బడ్జెట్ లో కష్టం. మొదటిది దాదాపు ఎప్పుడూ మెరుగు, ఎందుకంటే అది **నిజమైన మానవ ప్రవర్తన** నుంచి వస్తుంది.

"చాలా భాషలు, చాలా లిపులు ఉంటే?"

ఒక్కో లిపికి **వేరే index**. కారణం §11 – తెలుగు queries మరియు English queries ఒకే prefix ని పంచుకోవు, కాబట్టి ఒకే index లో ఉంచడం వల్ల లాభం లేదు, మరియు ప్రతి lookup పెద్ద candidate set ని తాకుతుంది. మరియు ఒక ఆచరణాత్మక సమస్య: **transliteration** – వాడుకరి "vaatavaranam" అని రాసి తెలుగు ఫలితాలు ఆశిస్తాడు. అది ఒక ప్రత్యేక mapping పొర, index కాదు.

"ఒక query ని అత్యవసరంగా తీసేయాలంటే?"

ఇది §5 యొక్క రెండు అంచెల design లో ఒక ఇబ్బంది: query **రెండు చోట్లా** ఉండొచ్చు. కాబట్టి ఒక మూడో నిర్మాణం కావాలి – ఒక **నిషేధ జాబితా**, ఇది suggest చివర్లో వర్తిస్తుంది మరియు index rebuild కోసం ఎదురుచూడదు. HLD Deep 01 §7 యొక్క అదే పాఠం: **తీసేయగలగడం ఒక design constraint, ఒక feature కాదు**.

"ఒక prefix కి వేర్వేరు ప్రాంతాల్లో వేర్వేరు సూచనలు?"

ఇది §7 యొక్క మృదువైన రూపం – cache key prefix + region . Regions కొన్ని వందలు మాత్రమే (users కోట్లు కాదు), కాబట్టి hit rate పెద్దగా పడదు. ఇదే **personalization కి సరైన మధ్య దారి**: వాడుకరి స్థాయిలో కాకుండా, ఒక చిన్న సమూహ స్థాయిలో.

"Typeahead index ని serving machines కి ఎలా పంపాలి?"

0.6 TB index ని ప్రతి build తర్వాత వందల machines కి పంపడం ఒక పెద్ద పని. ఆచరణ: **అంతర్గత rebuild** కాకుండా, index ని shards గా విభజించి, ఒక్కో shard ని దాని machines కి మాత్రమే పంపడం. మరియు rollout **క్రమంగా** ఉండాలి – కొత్త index lo bug ఉంటే అన్ని machines ఒకేసారి చెడిపోకూడదు.

16. ఏమి నేర్చుకున్నాం

1. ఎక్కడ సమస్య లేదో తెలియడం కూడా ఒక ఫలితం. Index **0.6 TB** (2 machines), distributed top-K **100%** ఖచ్చితం. ఆ రెండు కొలితలు interview lo 20 నిమిషాలు ఆదా చేస్తాయి.
2. తాజాదనం మరియు పరిపూర్ణత రెండు వేర్వేరు అవసరాలు. ఒక్క system తో రెండూ సాధించాలంటే **96x CPU**. రెండు systems తో అదే ఖర్చుతో **1 నిమిషం**.
3. పాత డేటా పెద్దది కానీ స్థిరం; కొత్త డేటా చిన్నది కానీ చంచలం. అందుకే చిన్నదానికి index అవసరం లేదు – ఒక linear scan చాలు.
4. Cache ని పెద్దది చేయడం key space సమస్యని పరిష్కరించదు. 3 మి → 300 మి entries, hit rate **0.2% నుంచి 0.2%**. ఏదీ పునరావృతం కానప్పుడు cache నిరుపయోగం.
5. **Personalization ఒక వడపోత కాదు, ఒక పునఃక్రమం**. ఖరీదైనది ("ఏవి సంబంధితం") అందరికీ ఒకటే; చవ్వైనది ("ఏది ముందు") ఒక్కొక్కరికీ వేరే.
6. తక్కువ సాక్ష్యంతో మంచి సమాధానం అసాధ్యం. రోజుకి 15 queries ఉన్న prefix కి top-10 lo **ఆరు రోజు మారతాయి**. అక్కడ సరైన పని **మౌనం** – మరియు అది calls lo **54%**.
7. **మౌనంగా తప్పు జవాబు ఇవ్వడం, జవాబు ఇవ్వకపోవడం కంటే ఘోరం**. నా index truncation "ఫలితాలు రావు" అని ఆశించాను; అది నిజానికి తప్పు లెక్కలతో ఫలితాలు ఇచ్చింది, మరియు అది **1,000 lo 1** సార్లే కనిపించింది.
8. రెండు అంచెల design యొక్క సాధారణ bug – కలిపాక ఖాళీ చెయ్యడం మర్చిపోవడం. అప్పుడు అవే queries రెండుసార్లు లెక్కవృతాయి: **19 ≠ 10**.

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ **problem** lo ప్రతి పరిష్కారం ఒక విభజన.

పాతది/కొత్తది వేరు చేయడం (§5). పంచుకునేది/వ్యక్తిగతమైనది వేరు చేయడం (§8). తెలిసినది/తెలియనిది వేరు చేయడం (§11).

మూడు సందర్భాల్లోనూ తప్పు ఏమిటంటే – రెండు వేర్వేరు స్వభావాలున్న వాటిని ఒకే యంత్రాంగంతో పరిష్కరించడానికి ప్రయత్నించడం.